Welcome to Code Crunch!

C1|Crunch Convos

WED 2-3PM Weekly | Spring 2025

Attendance

Earn a Certificate of Completion by attending at least 70% of the workshops

linktr.ee/CODE.CRUNCH

Unit #4

C1 | Linked Lists

Linked List

A **linked list** is a linear data structure where elements (called **nodes**) are stored in sequence, but each node contains:

- **Data:** The value of the node.
- **Pointer:** A reference (or link) to the next node in the sequence.

```
class Node:
    def __init__(self, val, next=None):
        self.val = val
        self.next = next
```

Visual

```
class Node:
    def __init__(self, val, next=None):
        self.val = val
        self.next = next

head = Node('A', Node('B', Node('C', Node('D'))))
```

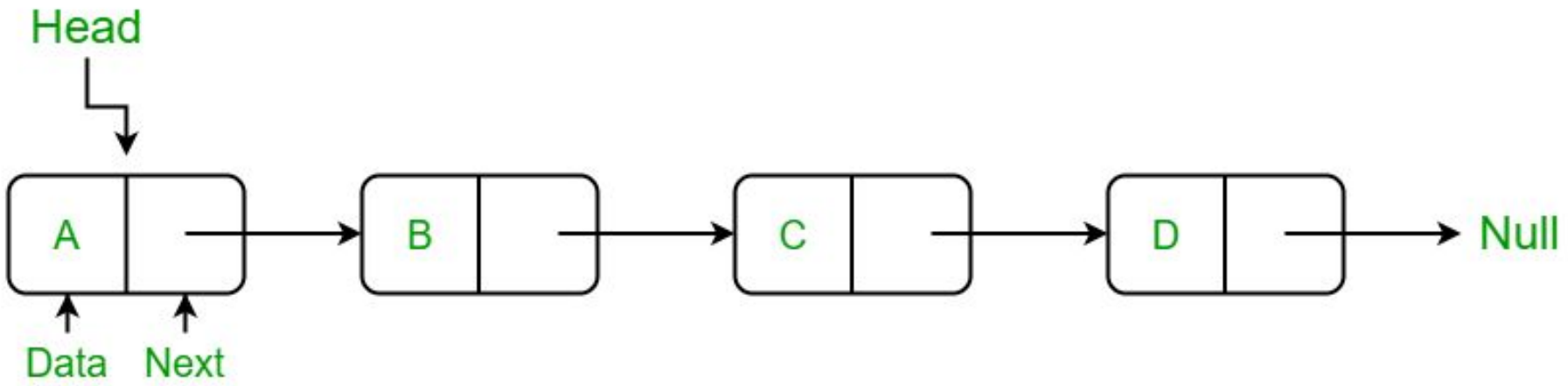

Image Source: GeeksforGeeks

Types of Linked Lists

1. **Singly Linked List:**
   - Each node has a reference to the next node.

2. **Doubly Linked List:**
   - Each node has references to both the next and previous nodes.

3. **Circular Linked List:**
   - The last node points back to the first node.

Linked List Operations

Insertion:

- Add a node at the beginning or end.

Deletion:

- Remove a node from a specific position.

Traversal:

- Iterate through the list to access or print values.

Search:

- Find if a value exists in the list.

Linked List Fundamentals

Space Complexity: $O(n)$ where n is the number of nodes.

Time Complexity:

- **Traversal:** $O(n)$ for searching or iterating through the list.
- **Insertion/Deletion:**
  - $O(1)$ if modifying at the head (or tail for doubly linked lists).
  - $O(n)$ if modifying at an arbitrary position.

Advantages:

- Dynamic size, no fixed allocation.
- Efficient insertions and deletions compared to arrays.

Disadvantages:

- No direct access to elements ($O(n)$ traversal is needed).

Linked List Practice

https://colab.research.google.com/drive/1fQXm7My1KfjpFtrsIb60IcqakBm54s7d?usp=sharing

Practice Problem (Leetcode #206)

Given the head of a singly linked list, reverse the list, and return the reversed list.

Example

Input: head = 1 -> 2 -> 3 -> 4 -> 5

Output: 5 -> 4 -> 3 -> 2 ->1

Attendance

linktr.ee/CODE.CRUNCH

Join us for the next sessions

Crunch Convos: WED 2PM - 3PM
Crunch Code: FRI 1PM - 2PM

RSVP: lu.ma/CODECRUNCH

Your feedback helps us improve. Share your thoughts!
Go to our website: go.fiu.edu/codecrunch